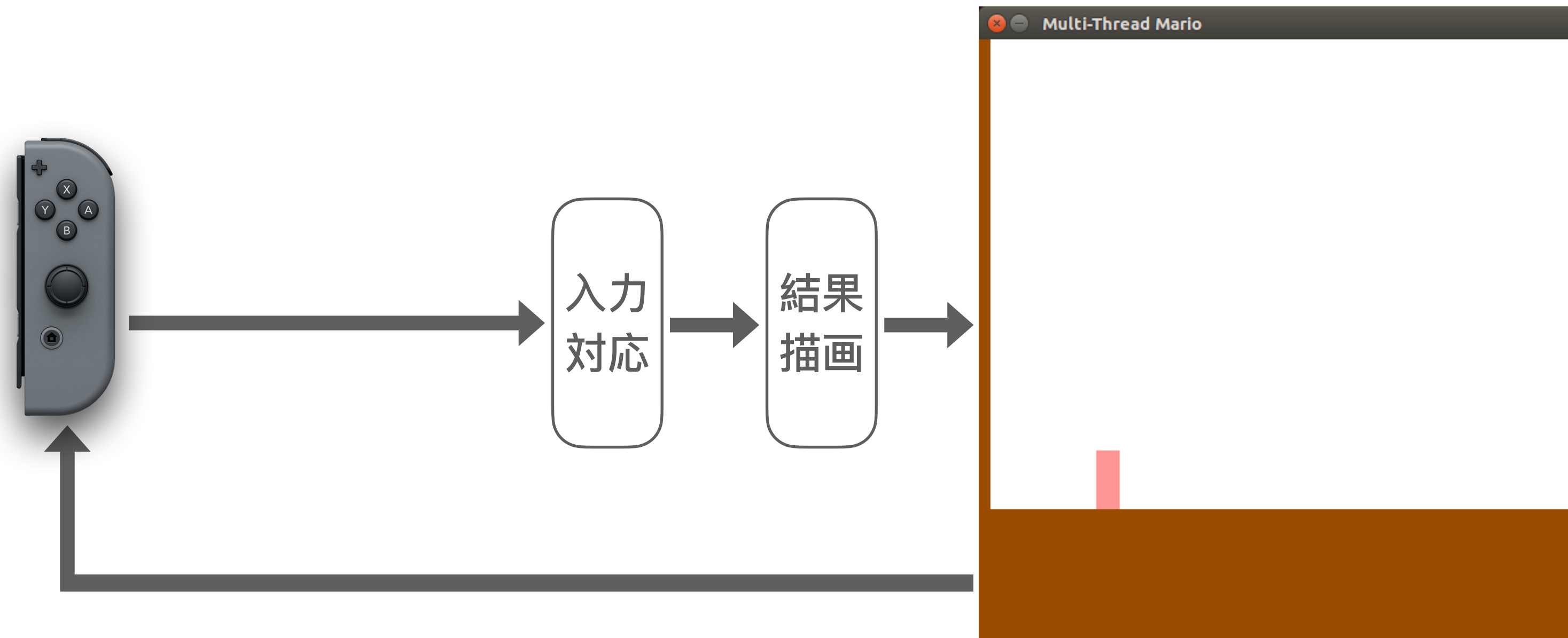


ソフトウェア設計及び実験

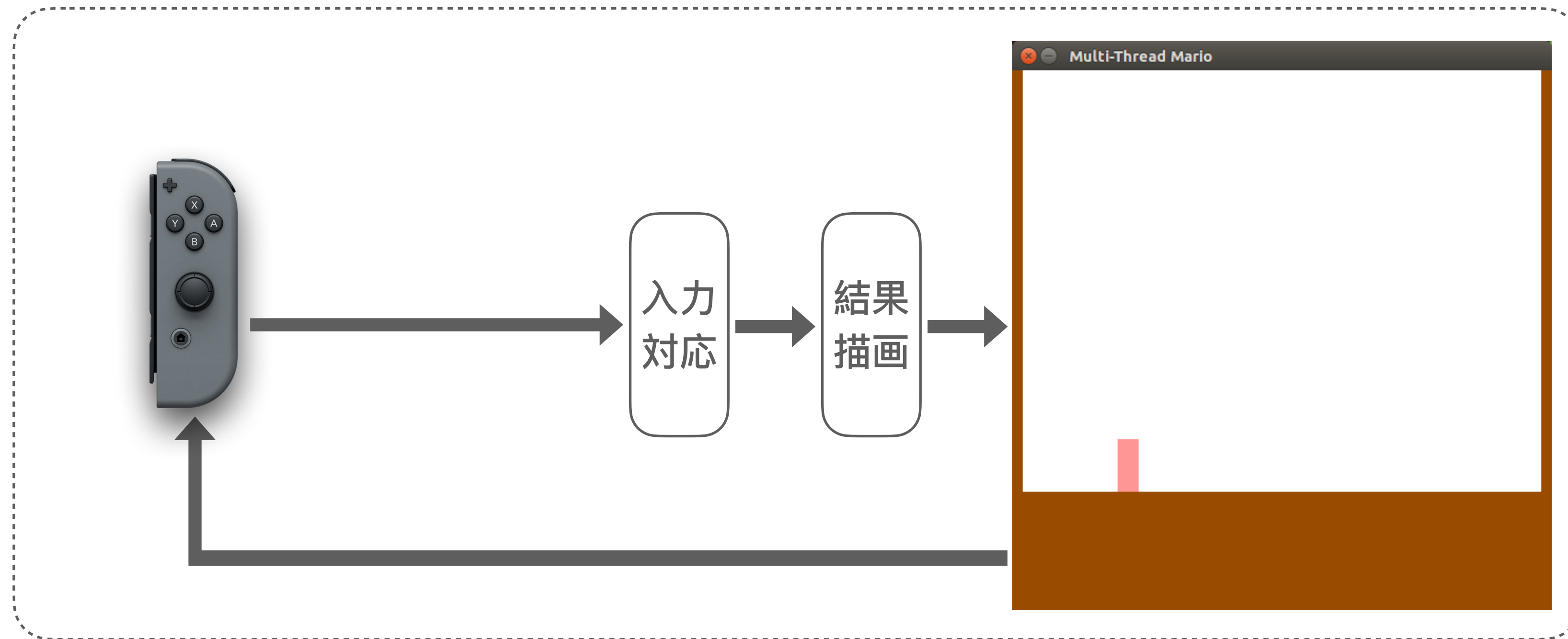
マルチスレッドプログラミング

担当：康 鑫

シングルスレッドプログラミング

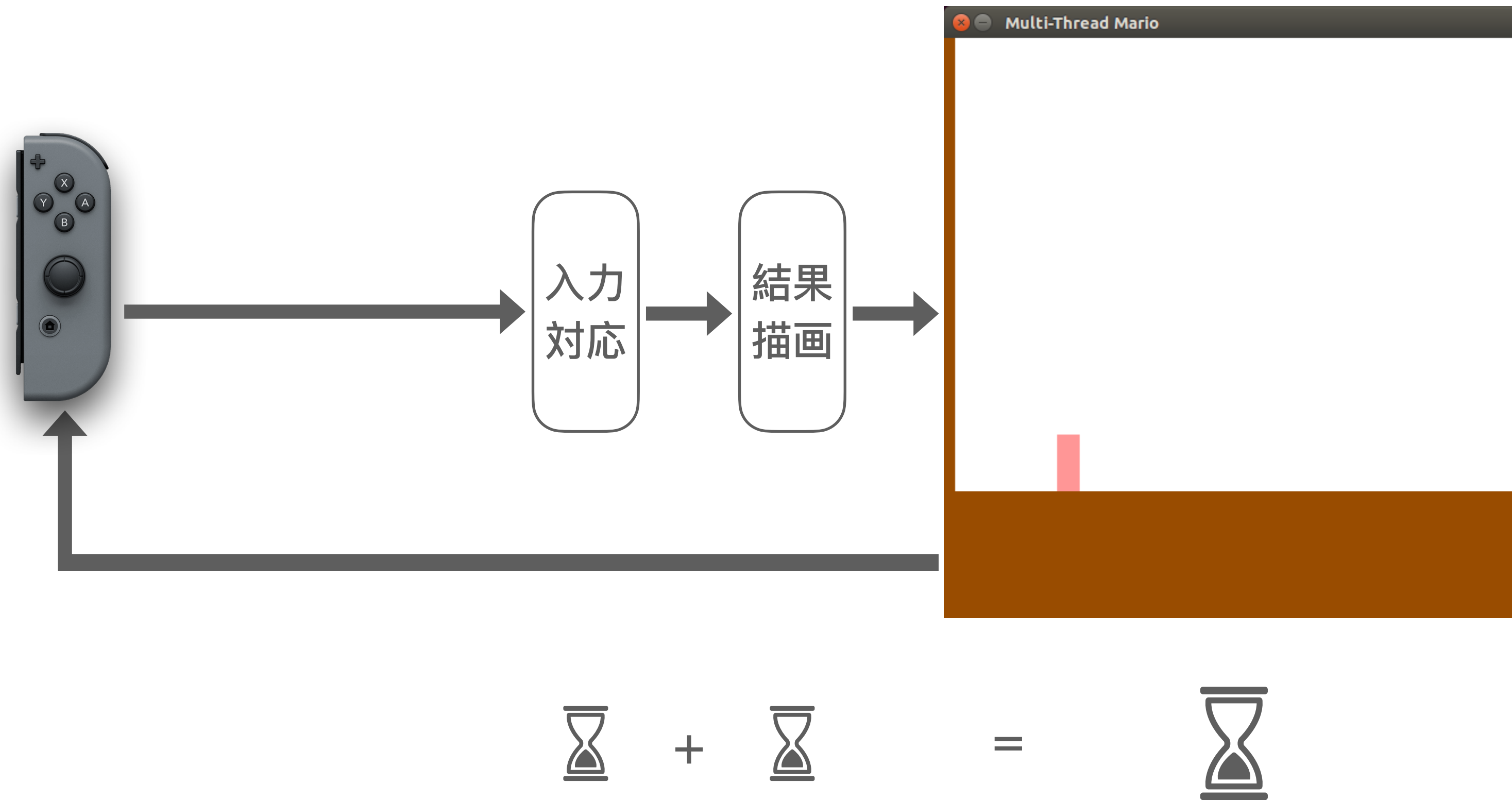


シングルスレッドプログラミング

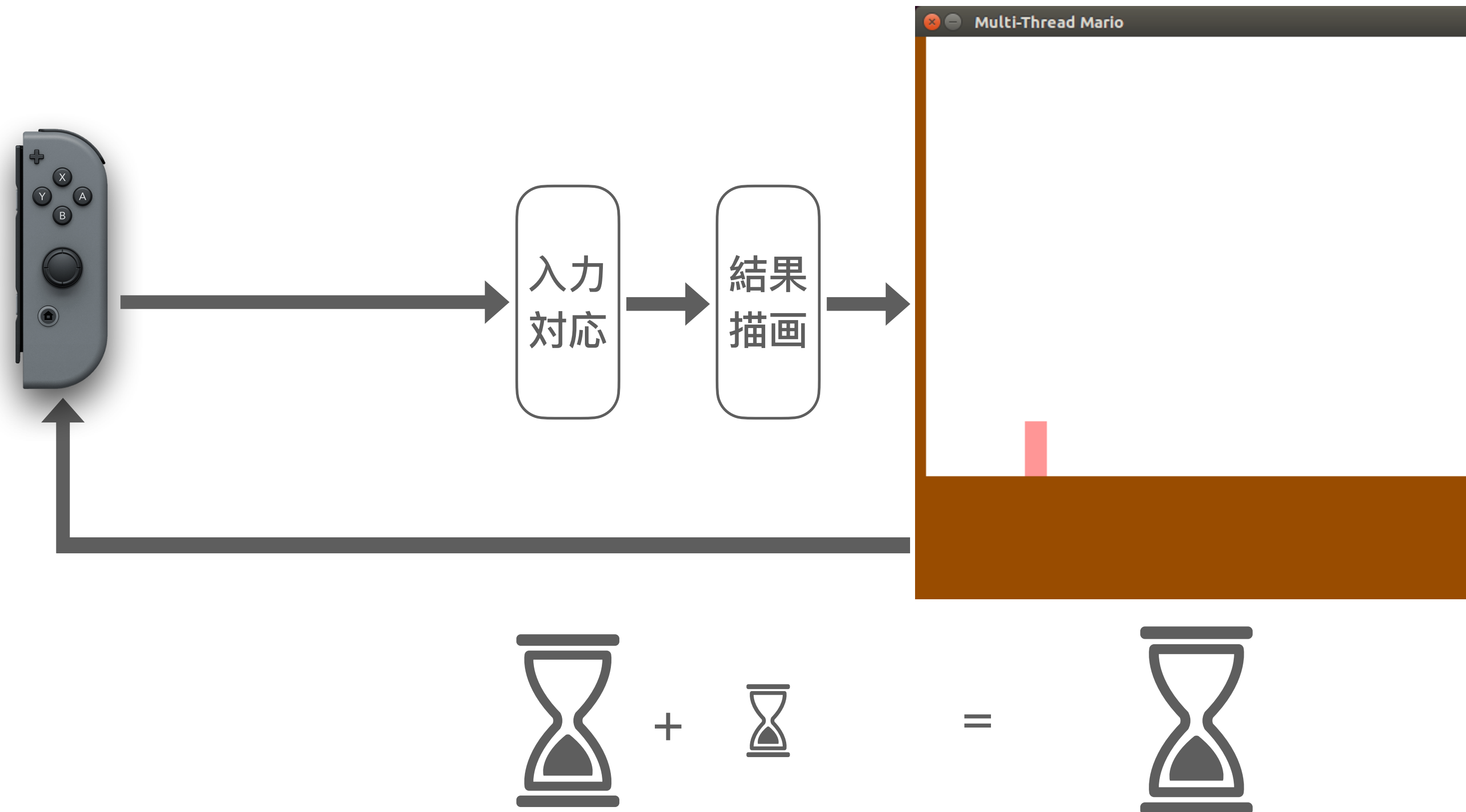


シングルスレッド

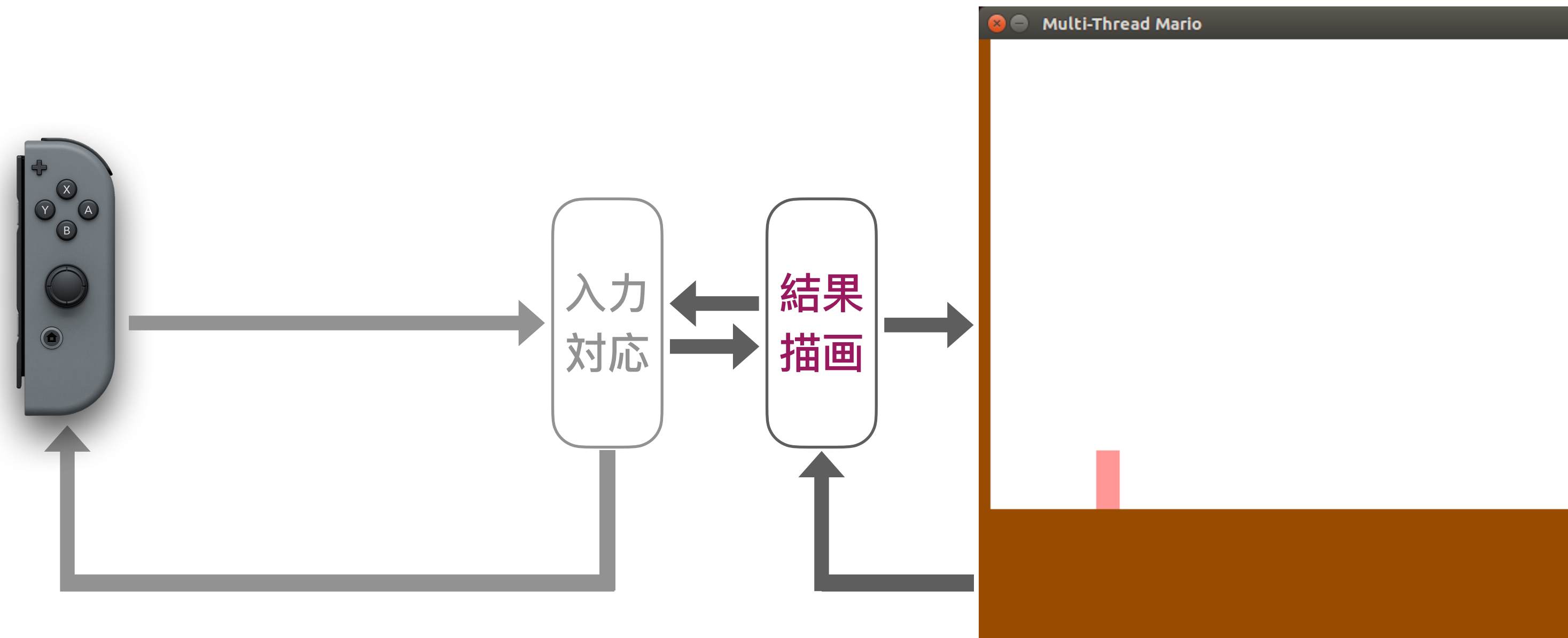
シングルスレッドプログラミング



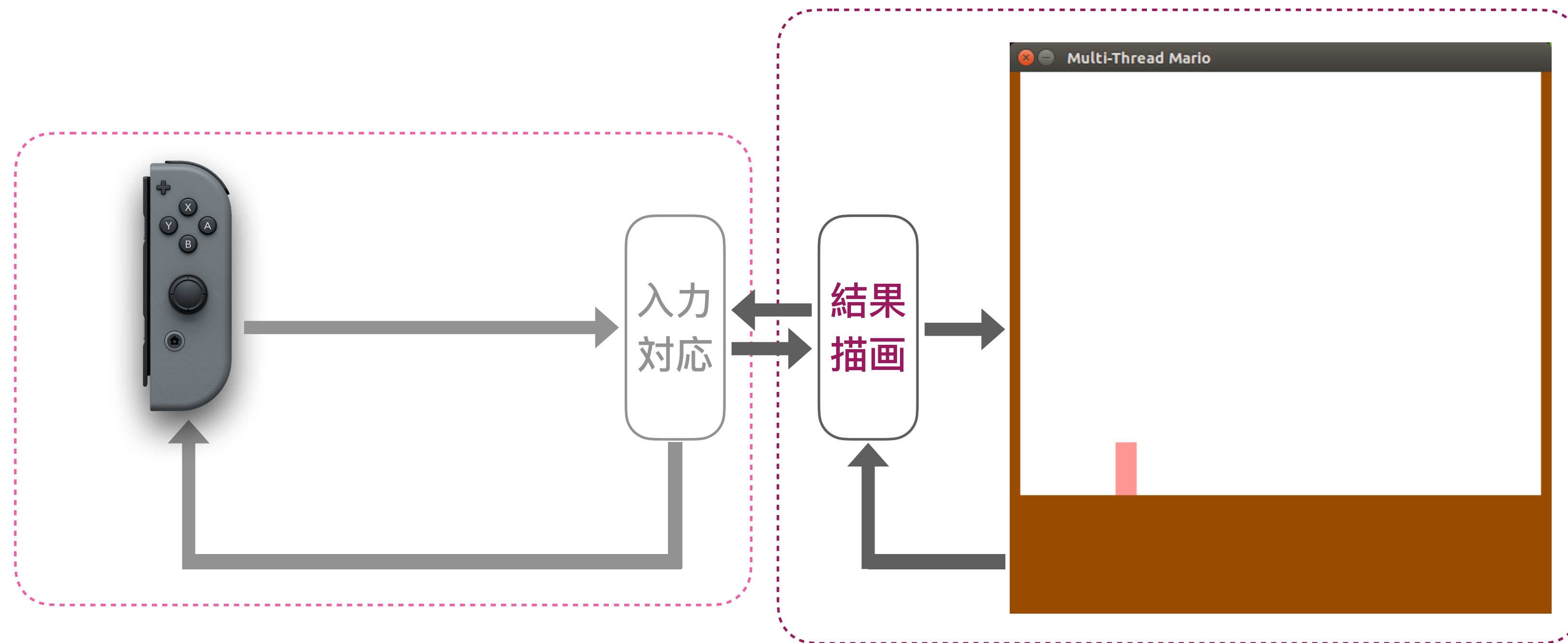
シングルスレッドプログラミング



マルチスレッドプログラミング



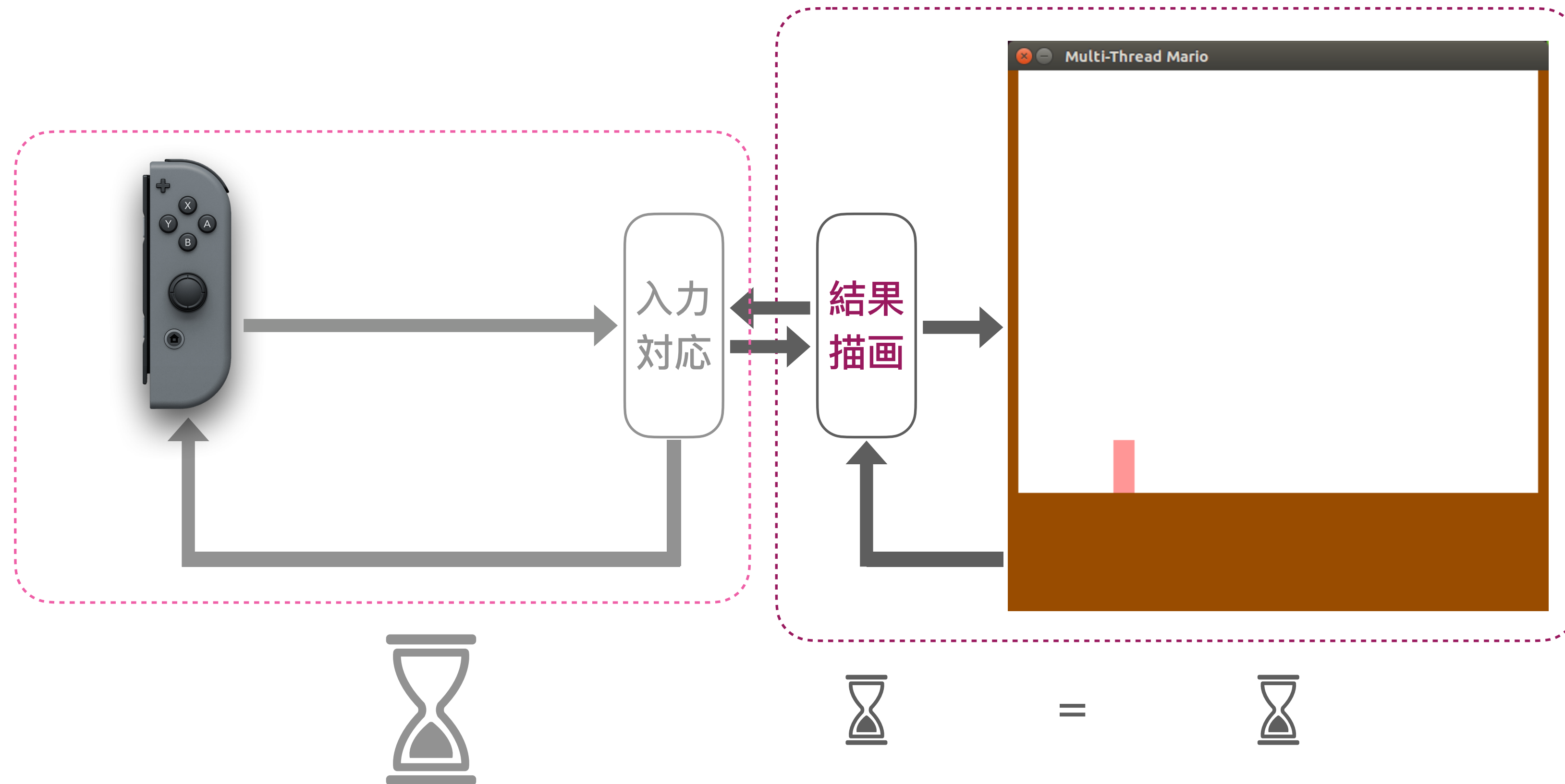
マルチスレッドプログラミング



入力対応スレッド

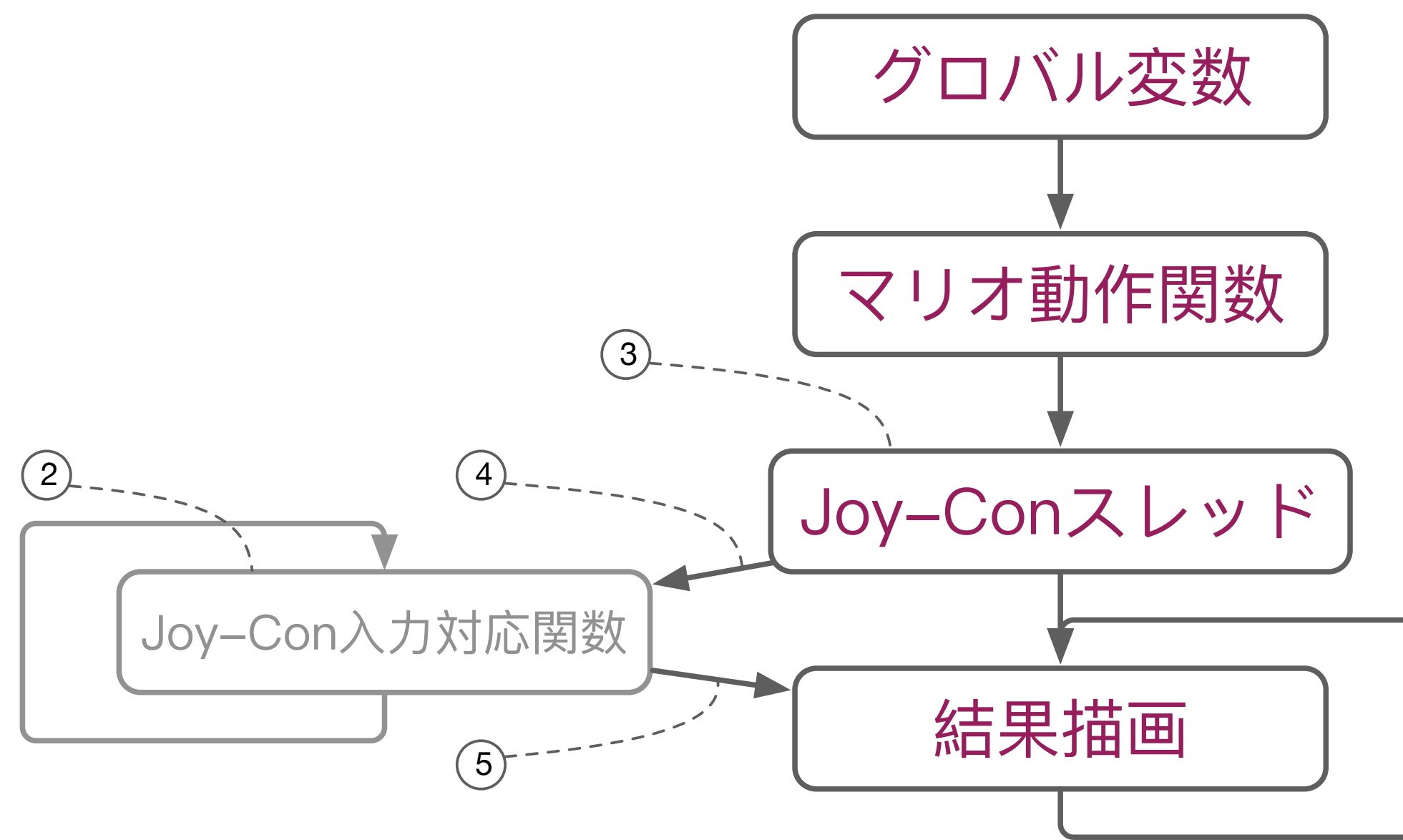
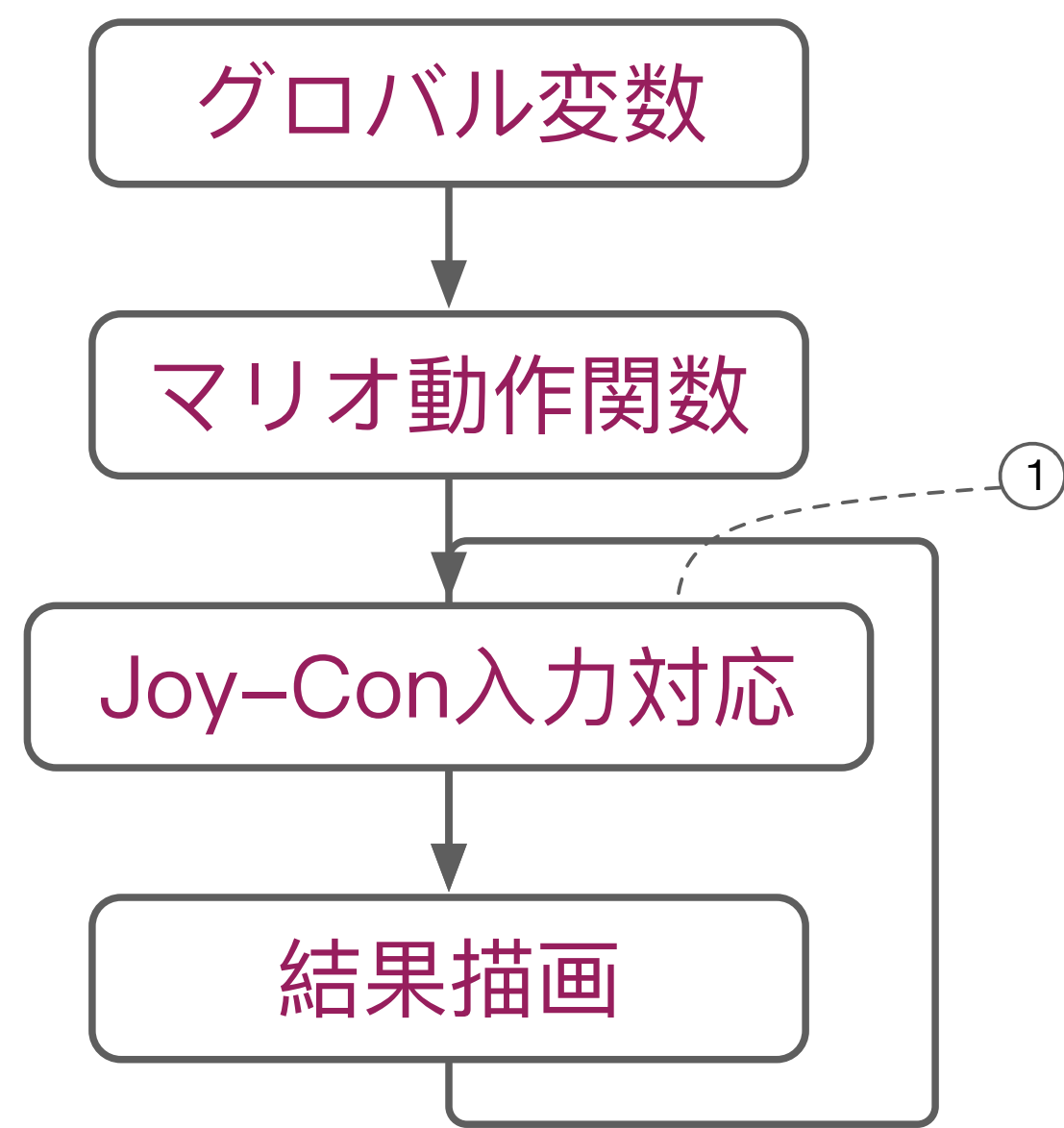
メインスレッド
(元々のシングルスレッド)

マルチスレッドプログラミング



プログラミングの手順

- ① 「Joy-Con入力対応」 移動
- ② 「Joy-Con入力対応関数」 作成
- ③ 「Joy-Conスレッド」 宣言
- ④ 「Joy-Con入力対応関数」 の開始
- ⑤ 「Joy-Con入力対応関数」 の終了

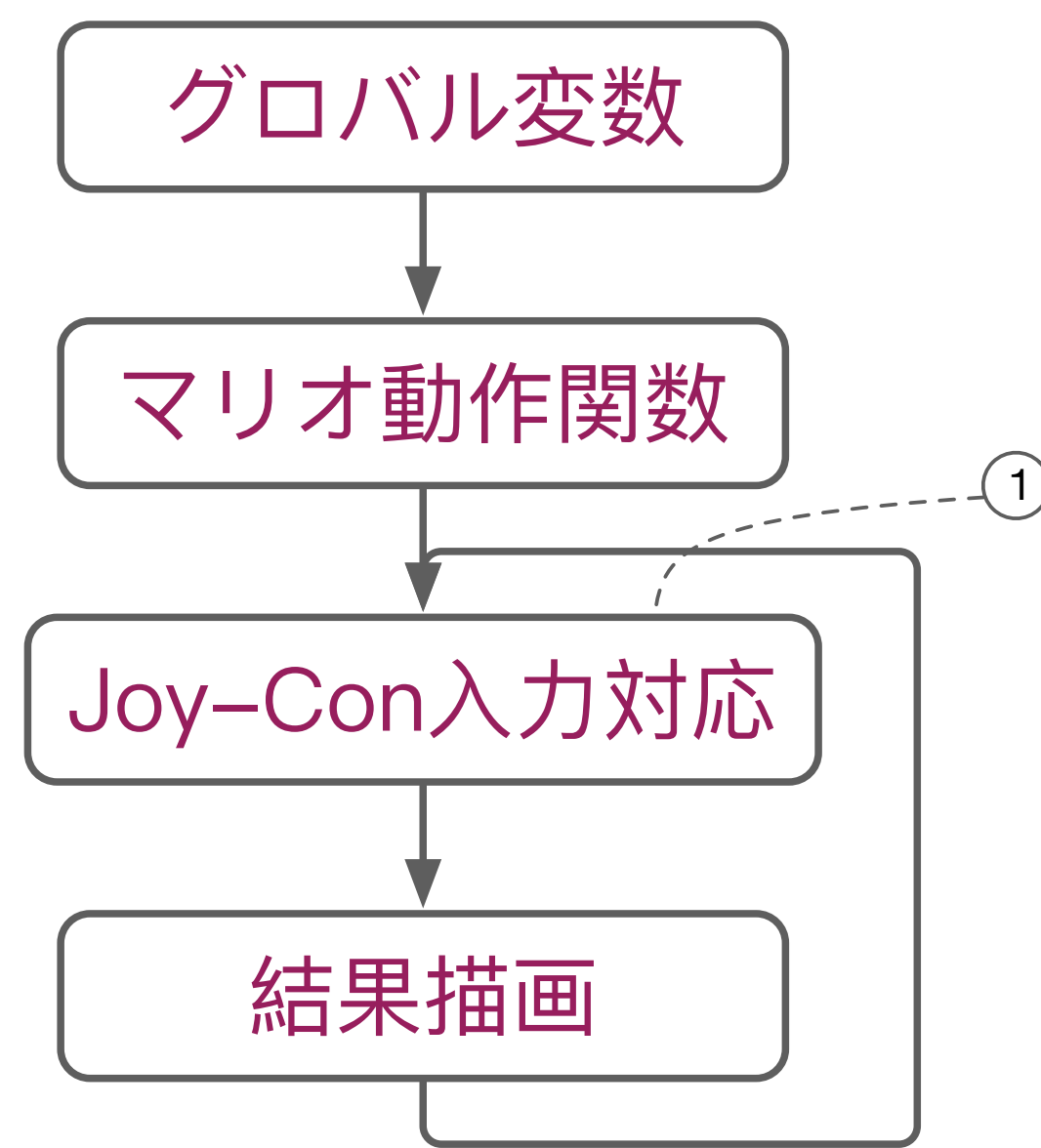


sdl2_thread.c

プログラミングの手順

① 「Joy-Con入力対応」 移動

② 「Joy-Con入力対応関数」 作成



```
////////////////////////////////////  
// STEP. 1                               //  
// STEP_1_BEGINからSTEP_1_ENDまでの //  
// コードをStep. 2で指示されたところ //  
// に移動する                               //  
////////////////////////////////////  
  
// ***** STEP_1_BEGIN (Joy-Conのイベント処理の初め) *****  
// Joy-Conの状態を取得  
joycon_get_state(&jc);  
...  
// ***** STEP_1_END (Joy-Conのイベント処理の終わり) *****
```

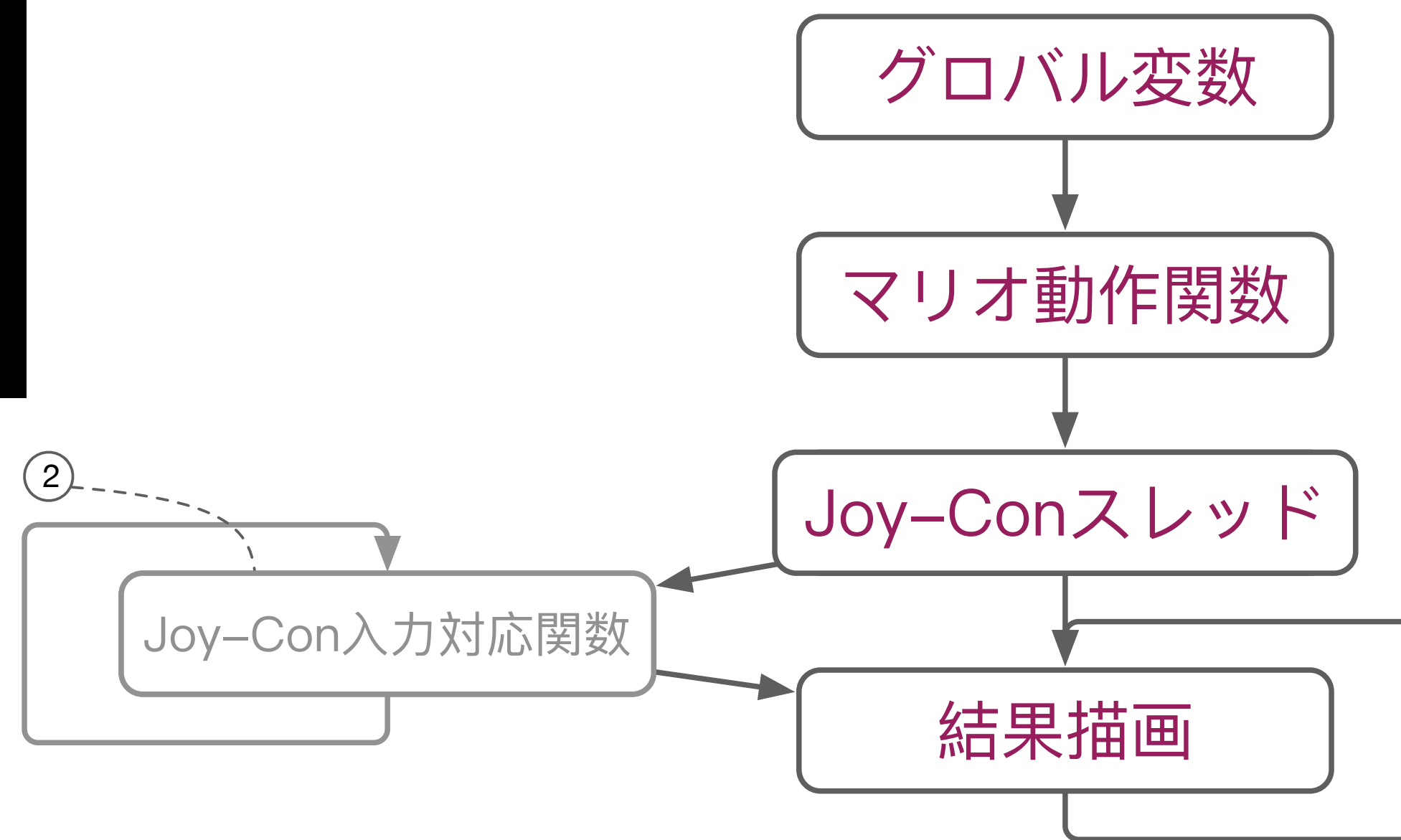
プログラミングの手順

```
// Joy-Conの状態を監視する関数
int joy_func(void *args)
{
    while(1)
    {
        //////////////////////////////////////
        // STEP. 2                               //
        // STEP_1で指示されたコードを           //
        // STEP_2_BEGINからSTEP_2_ENDまでに     //
        // に移動する                             //
        //////////////////////////////////////

        // ***** STEP_2_BEGIN (Joy-Conのイベント処理の初め) *****
        // ***** STEP_2_END (Joy-Conのイベント処理の終わり) *****
    }
    return 0;
}
```

- joy_func: Joy-Con入力対応関数
- args: void *型の引数
(実際の引数の型に変更できる)
- 返回值: 通常は0

- ① 「Joy-Con入力対応」 移動
- ② 「Joy-Con入力対応関数」 作成
- ③ 「Joy-Conスレッド」 宣言
- ④ 「Joy-Con入力対応関数」 の開始
- ⑤ 「Joy-Con入力対応関数」 の終了



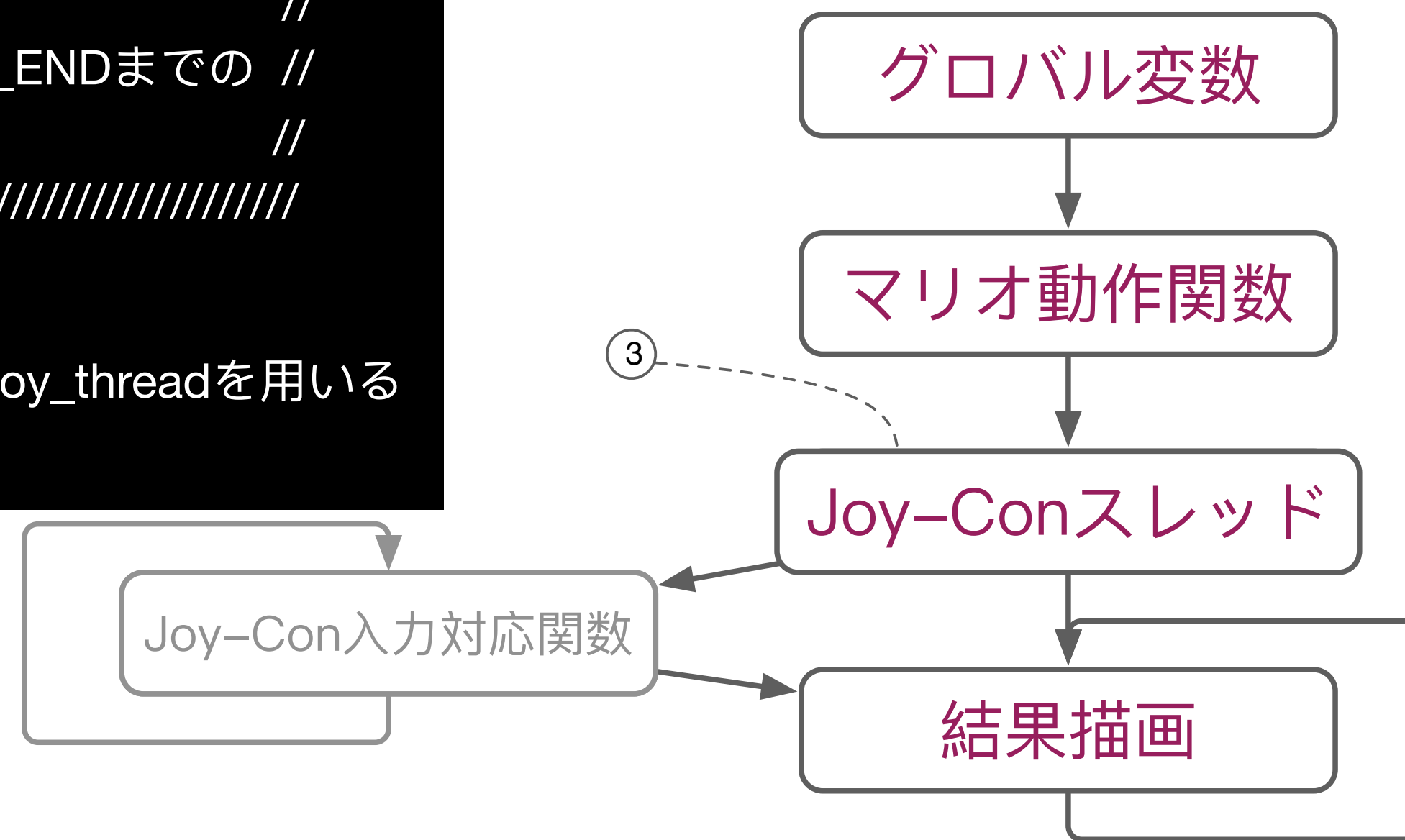
sdl2_thread.c

プログラミングの手順

SDL_Thread: SDLスレッドの型

```
////////////////////////////////////  
// STEP. 3                               //  
// STEP_3_BEGINからSTEP_3_ENDまでの //  
// コードのコメントを外す           //  
////////////////////////////////////  
  
// ***** STEP_3_BEGIN *****  
// SDL_Thread * joy_thread; // joy_threadを用いる  
// ***** STEP_3_END *****
```

- ① 「Joy-Con入力対応」 移動
- ② 「Joy-Con入力対応関数」 作成
- ③ 「Joy-Conスレッド」 宣言
- ④ 「Joy-Con入力対応関数」 の開始
- ⑤ 「Joy-Con入力対応関数」 の終了



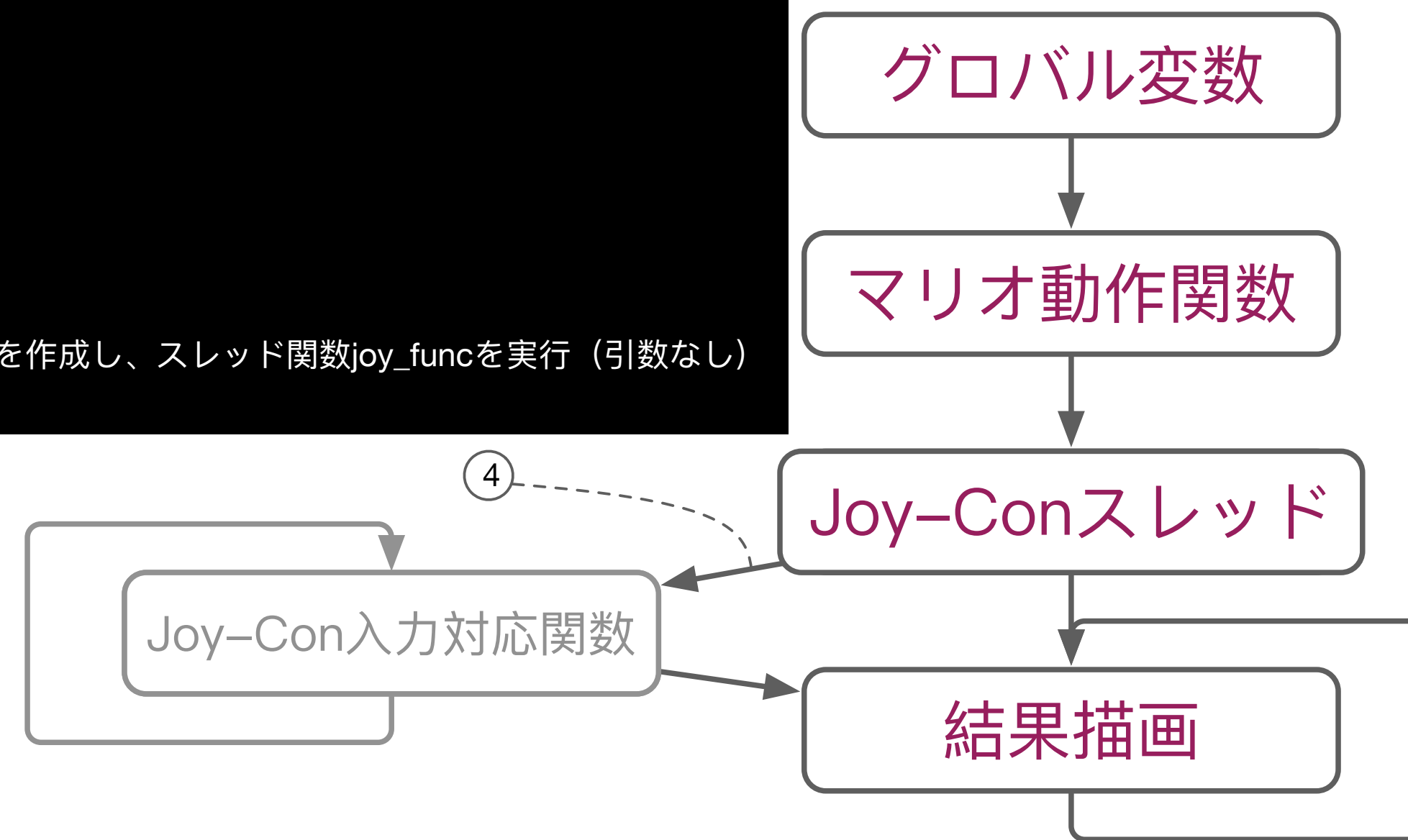
sdl2_thread.c

プログラミングの手順

- SDL_CreateThread: スレッド作成・実行
- 引数 1 (joy_func): ②で作成した関数名
- 引数 2 ("joy_thread"): スレッドの名前
- 引数 3 (NULL): ②で作成した関数の引数
- 返値(joy_thread): 作成されたスレッド③に

- ① 「Joy-Con入力対応」 移動
- ② 「Joy-Con入力対応関数」 作成
- ③ 「Joy-Conスレッド」 宣言
- ④ 「Joy-Con入力対応関数」 の開始
- ⑤ 「Joy-Con入力対応関数」 の終了

```
////////////////////////////////////  
// STEP. 4                               //  
// STEP_4_BEGINからSTEP_4_ENDまでの //  
// コードのコメントを外す           //  
////////////////////////////////////  
  
// ***** STEP_4_BEGIN *****  
// スレッドを作成・実行  
// joy_thread = SDL_CreateThread(joy_func, "joy_thread", NULL); // joy_threadを作成し、スレッド関数joy_funcを実行（引数なし）  
// ***** STEP_4_END *****
```



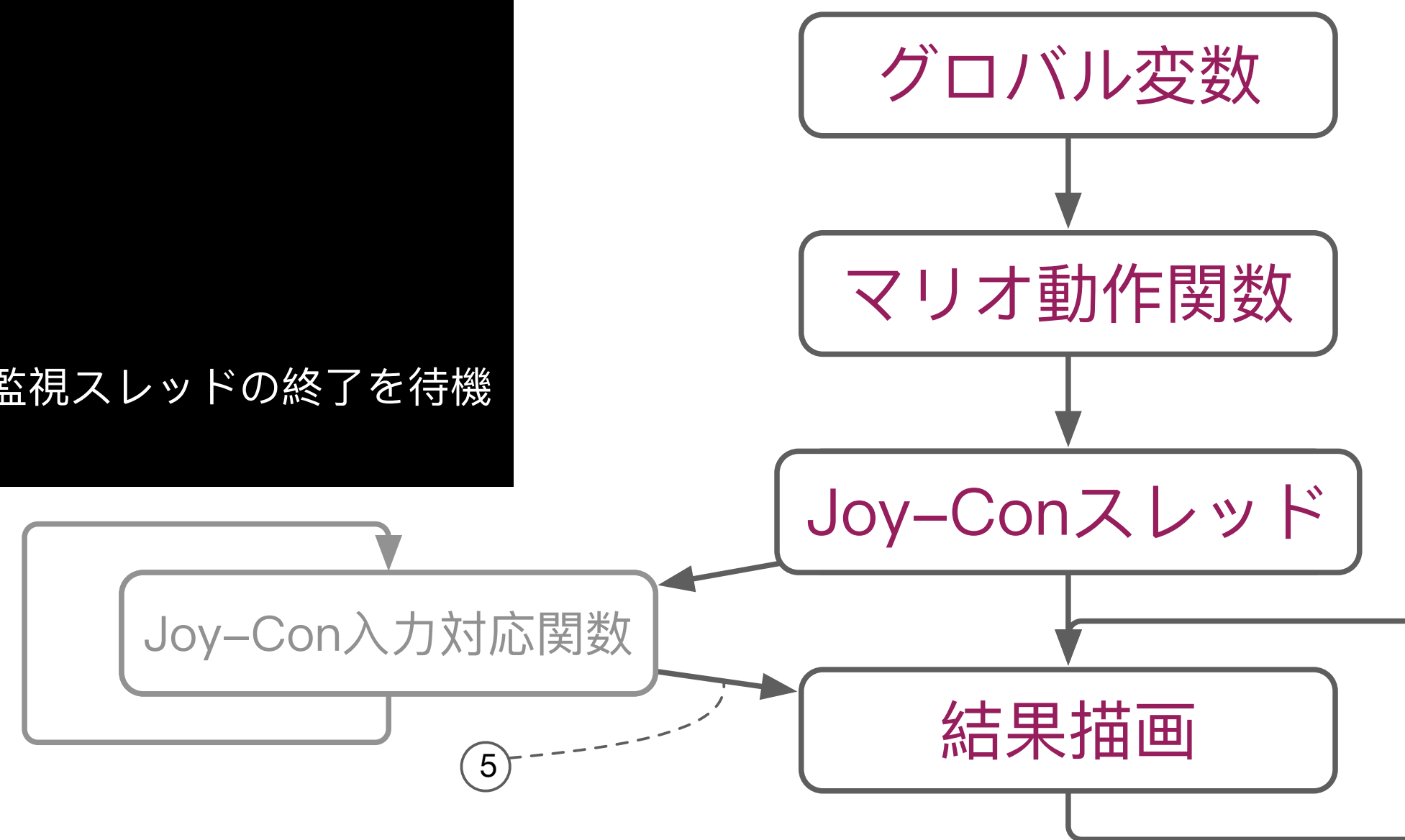
sdl2_thread.c

プログラミングの手順

- SDL_WaitThread: スレッドが終了するのを待つ
- 引数 1 (joy_thread): ③で宣言されたスレッド
- 引数 2 (NULL): スレッドの状態 (通常はNULL)
- 返回值: なし

- ① 「Joy-Con入力対応」 移動
- ② 「Joy-Con入力対応関数」 作成
- ③ 「Joy-Conスレッド」 宣言
- ④ 「Joy-Con入力対応関数」 の開始
- ⑤ 「Joy-Con入力対応関数」 の終了

```
////////////////////////////////////  
// STEP. 5                                //  
// STEP_5_BEGINからSTEP_5_ENDまでの //  
// コードのコメントを外す          //  
////////////////////////////////////  
  
// ***** STEP_5_BEGIN *****  
// SDL_WaitThread(joy_thread, NULL); // Joy-Con監視スレッドの終了を待機  
// ***** STEP_5_END *****
```



sdl2_thread.c

プログラミングのまとめ

- 目的：効率向上
- 方法
 - 非効率部分のコードを見付ける
 - スレッド関数に移動
 - スレッド変数を宣言
 - スレッドの開始
 - メインスレッドで他のコードを実行
 - スレッドの終了

実行してみよう

- コード修正

「sdl2_thread.c」をStep. 1~5に従って修正

- コンパイル

「make」で

- 実行

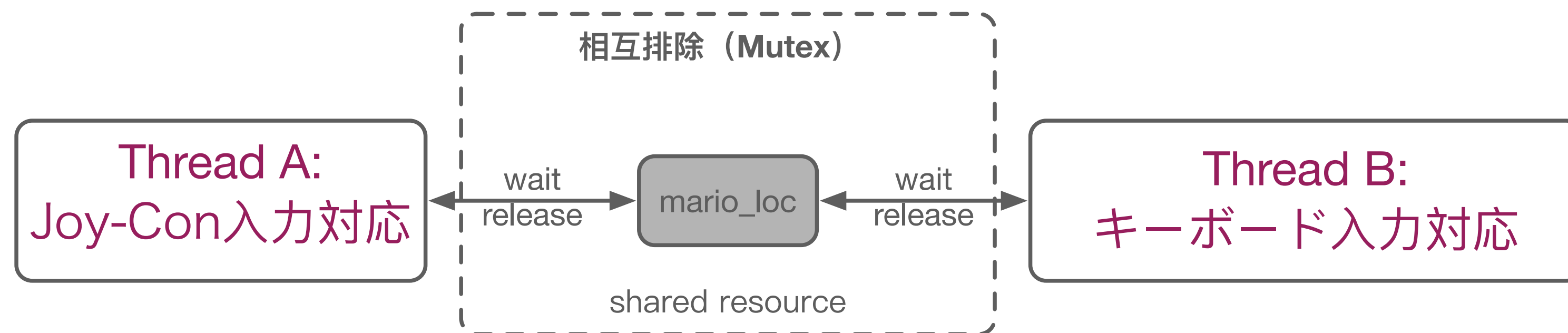
./sdl2_thread

相互排除

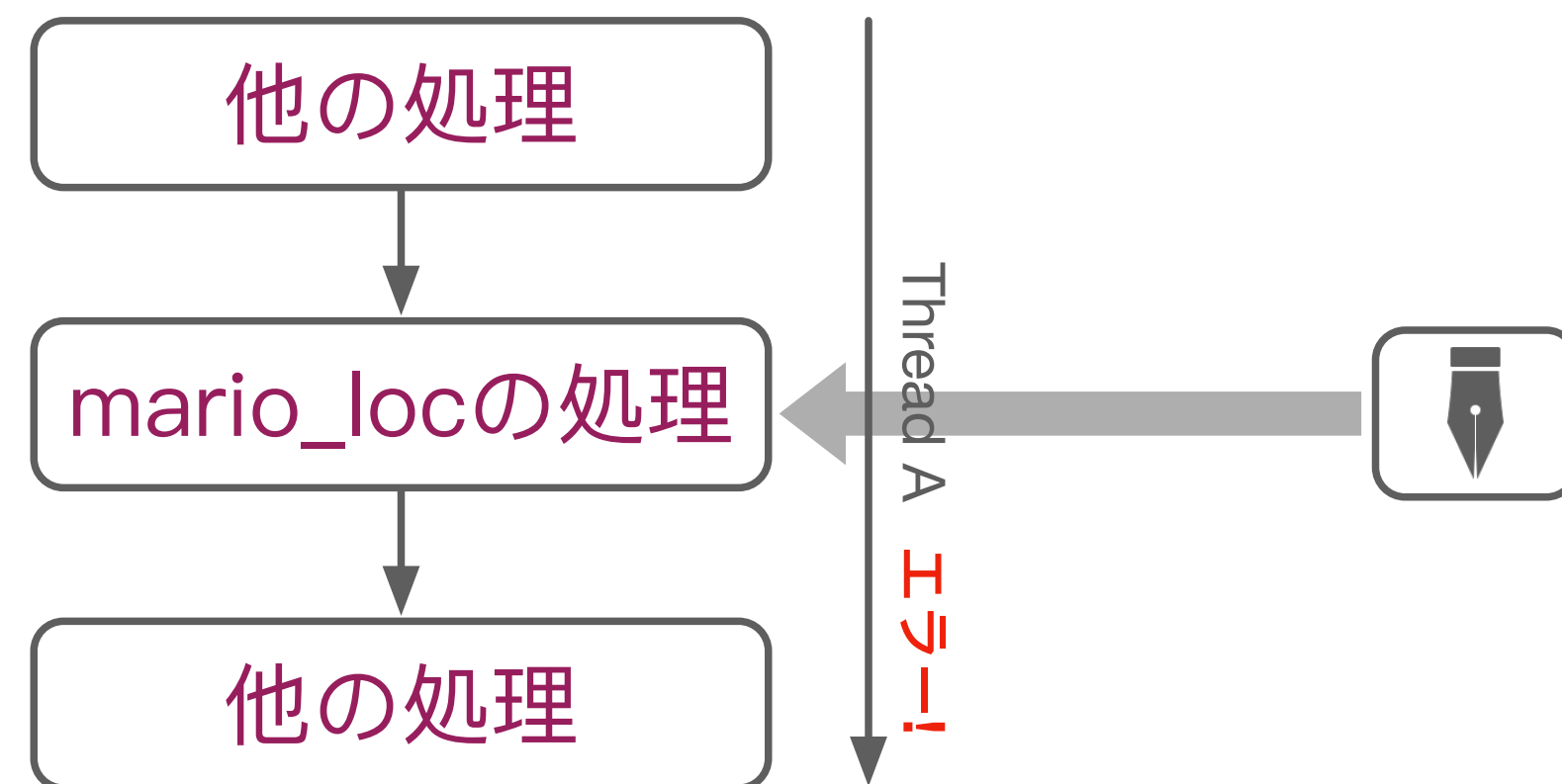
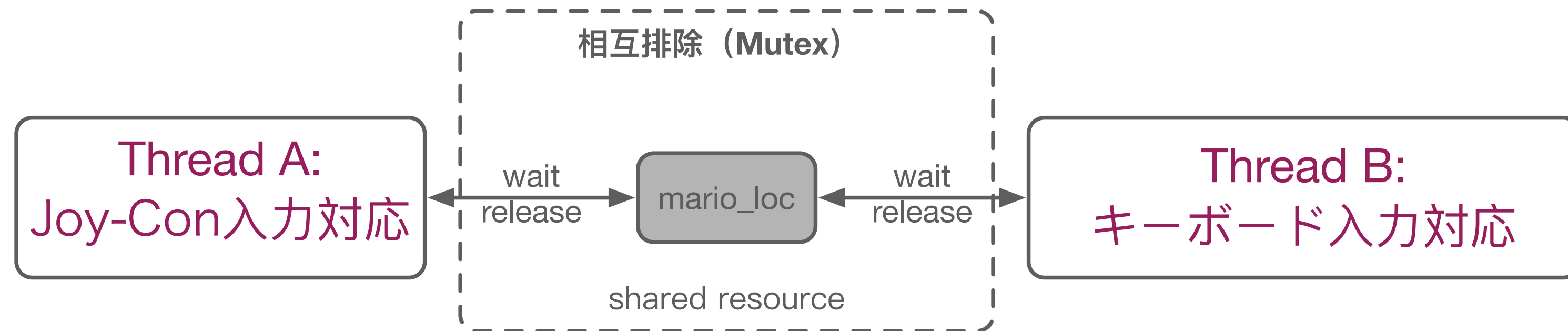
- 2つ以上のスレッドが同時に1つの変数値を変化 (✗)
- 例: Joy-Conとキーボード2つのデバイスで、マリオをコントロール

mario_locはThread Aに利用される時、Thread Bが一時中止

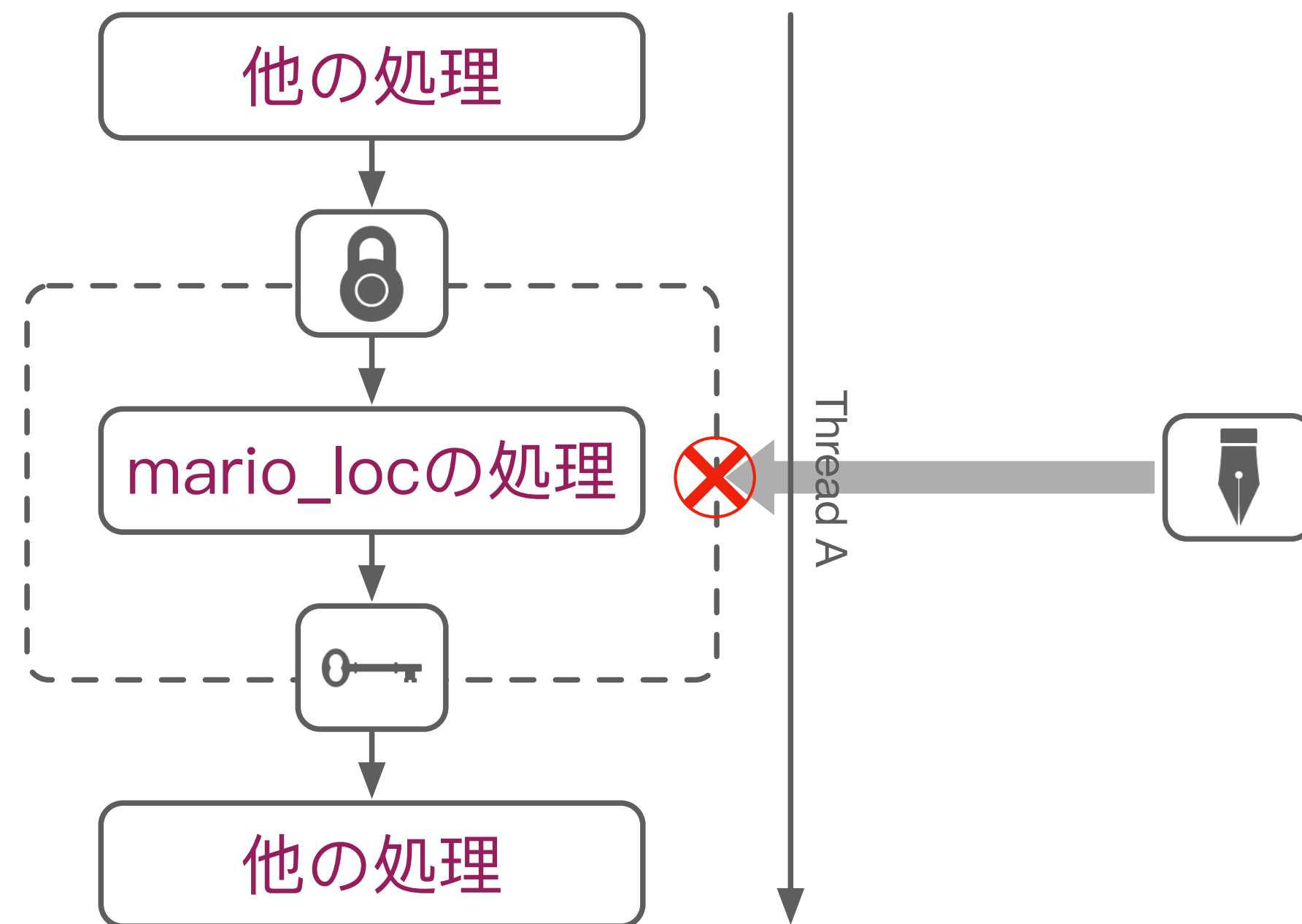
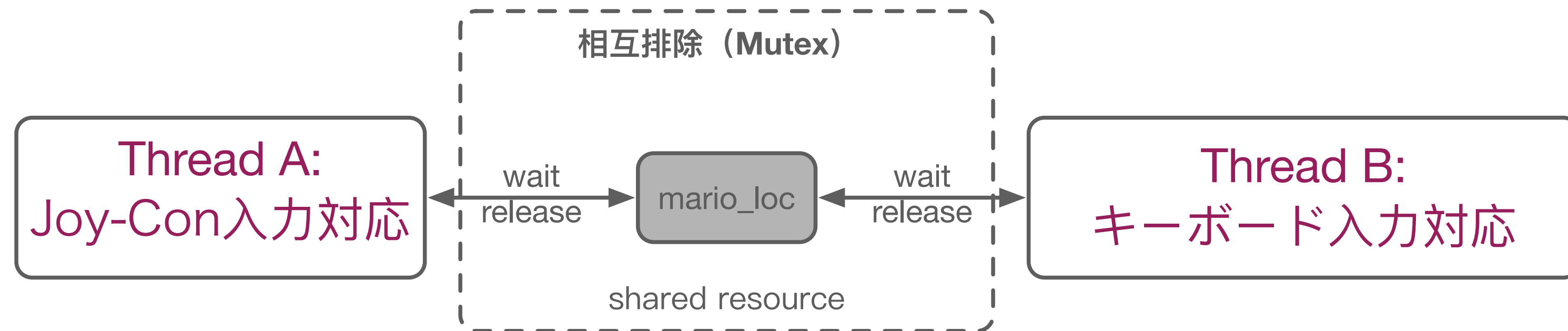
mario_locはThread Bに利用される時、Thread Aが一時中止



相互排除

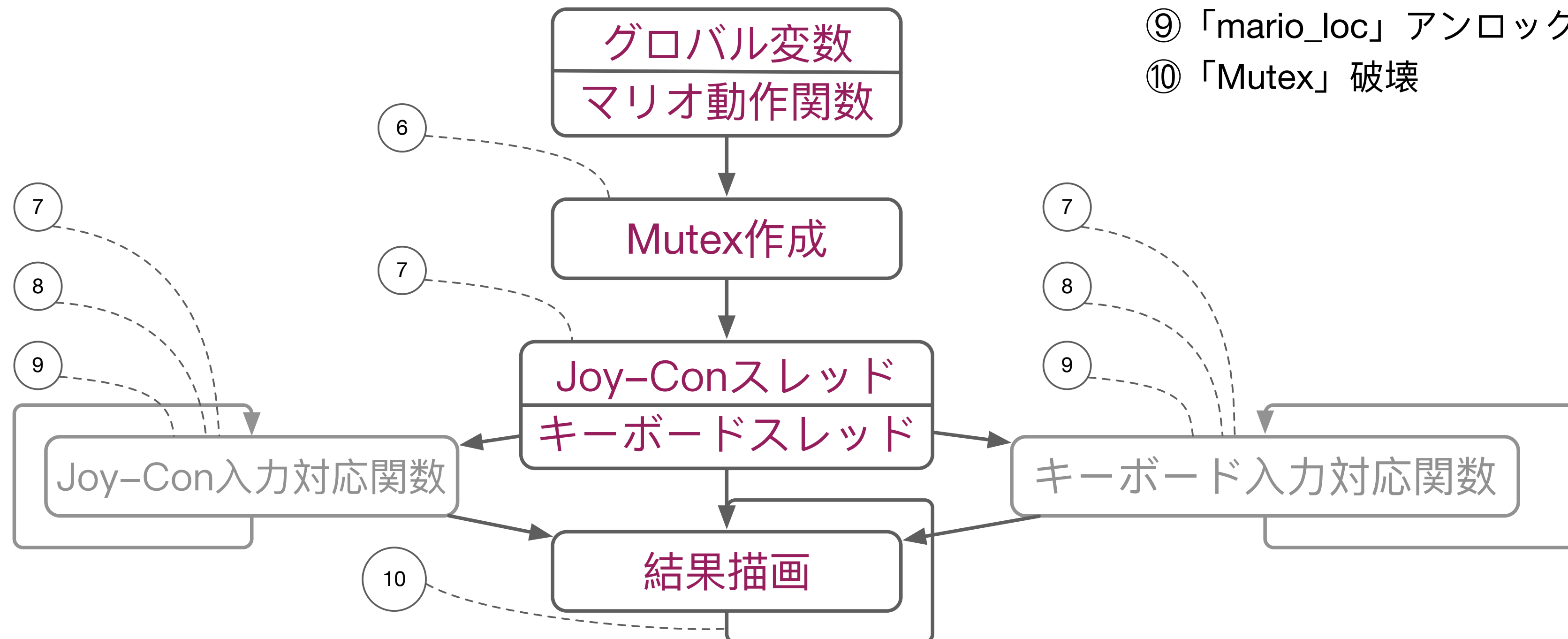


相互排除



プログラミングの手順

- ⑥ 「Mutex」 作成
- ⑦ 「スレッド関数の引数」 作成
- ⑧ 「mario_loc」 ロック
- ⑨ 「mario_loc」 アンロック
- ⑩ 「Mutex」 破壊



プログラミングの手順

```

////////////////////////////////////
// step. 6
// step_6_beginからstep_6_endまでの
// コードのコメントを外す
////////////////////////////////////

// ミューテックスの作成
// ***** step_6_begin *****
// sdl_mutex *mtx = sdl_createmutex();
// ***** step_6_end *****

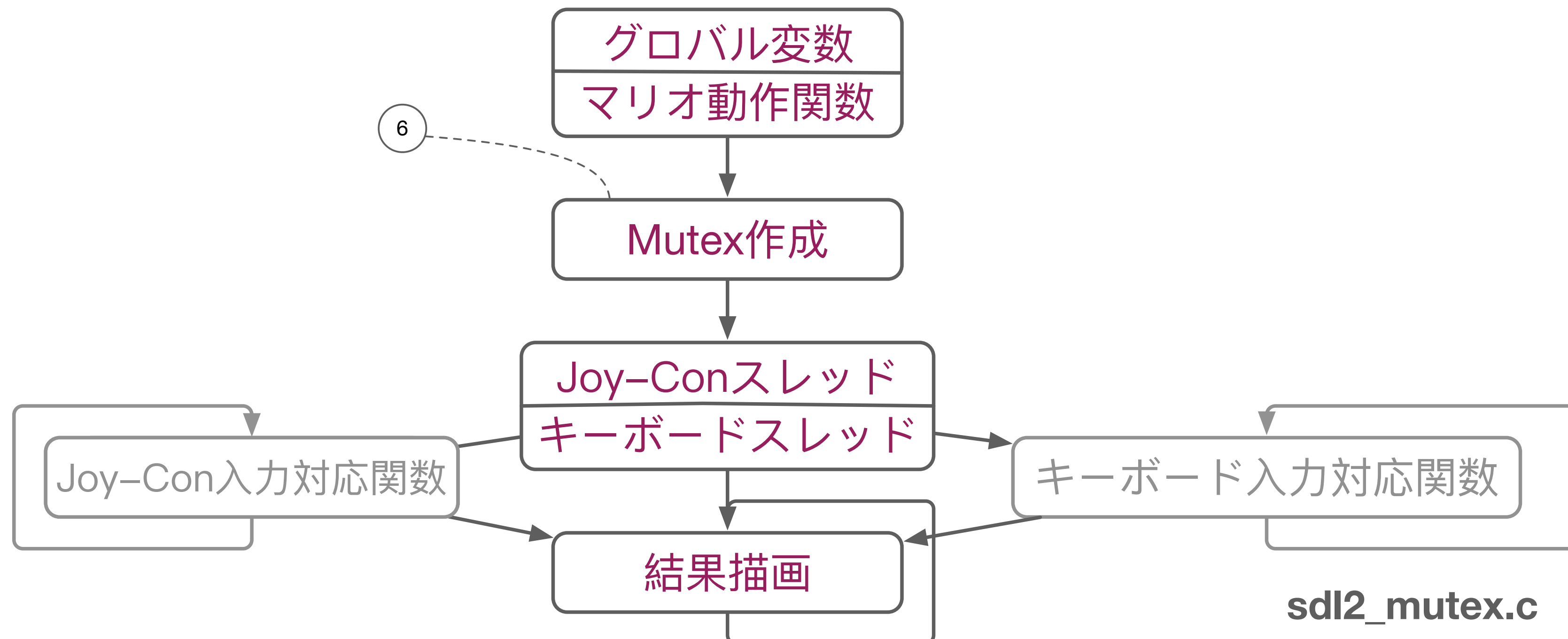
```

- **SDL_Mutex**: SDL相互排除の型
- **SDL_CreateMutex**: Mutex作成

⑥ 「Mutex」 作成

⑦「スレッド関数の引数」作成

ロック
アンロック
壊



プログラミングの手順

```
// Joy-Conの状態を監視する関数
int joy_func(void *args)
{
    //////////////////////////////////////
    // STEP. 7                                //
    // STEP_7_BEGINからSTEP_7_ENDまでの//
    // コードのコメントを外す                //
    //////////////////////////////////////

    // ***** STEP_7_BEGIN *****
    // SDL_mutex *mtx = (SDL_mutex *)args; // ミューテックスを取得
    // ***** STEP_7_END *****

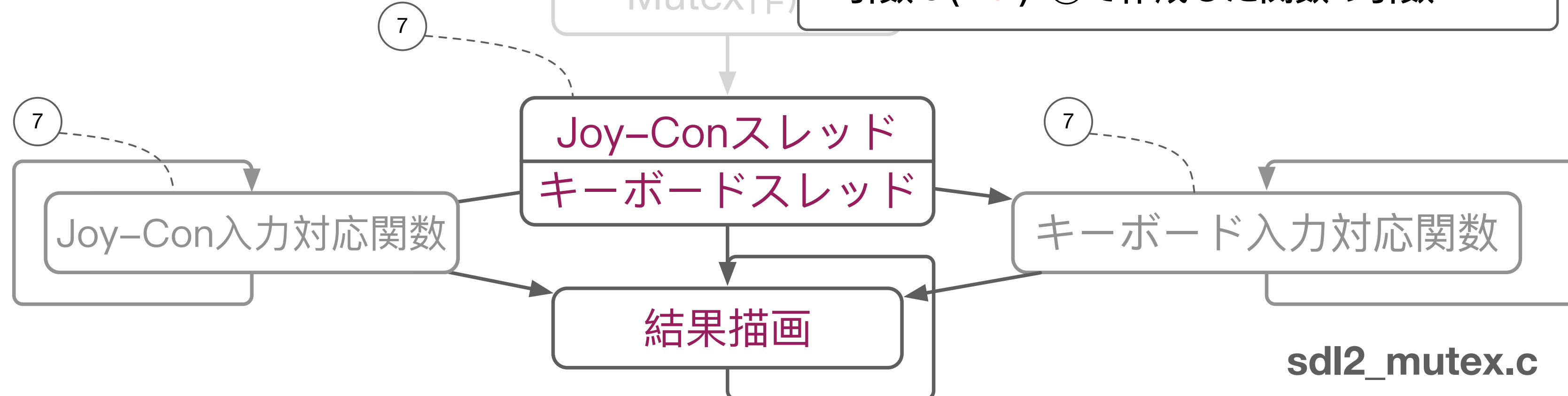
    ...
}
```

```
// Joy-Conの状態を監視するスレッドの作成
// joy_thread = SDL_CreateThread(joy_func, "joy_thread", mtx);
```

- ⑥ 「Mutex」 作成
- ⑦ 「スレッド関数の引数」 作成
- ⑧ 「mario_loc」 ロック
- ⑨ 「mario_loc」 アンロック
- ⑩ 「mario_loc」 破壊

- args: void *型の引数
(SDL_mutex型に変更)

- `SDL_CreateThread`: スレッド作成・実行
- 引数 1 (`joy_func`): ②で作成した関数名
- 引数 2 ("`joy_thread`"): スレッドの名前
- 引数 3 (`mtx`): ②で作成した関数の引数

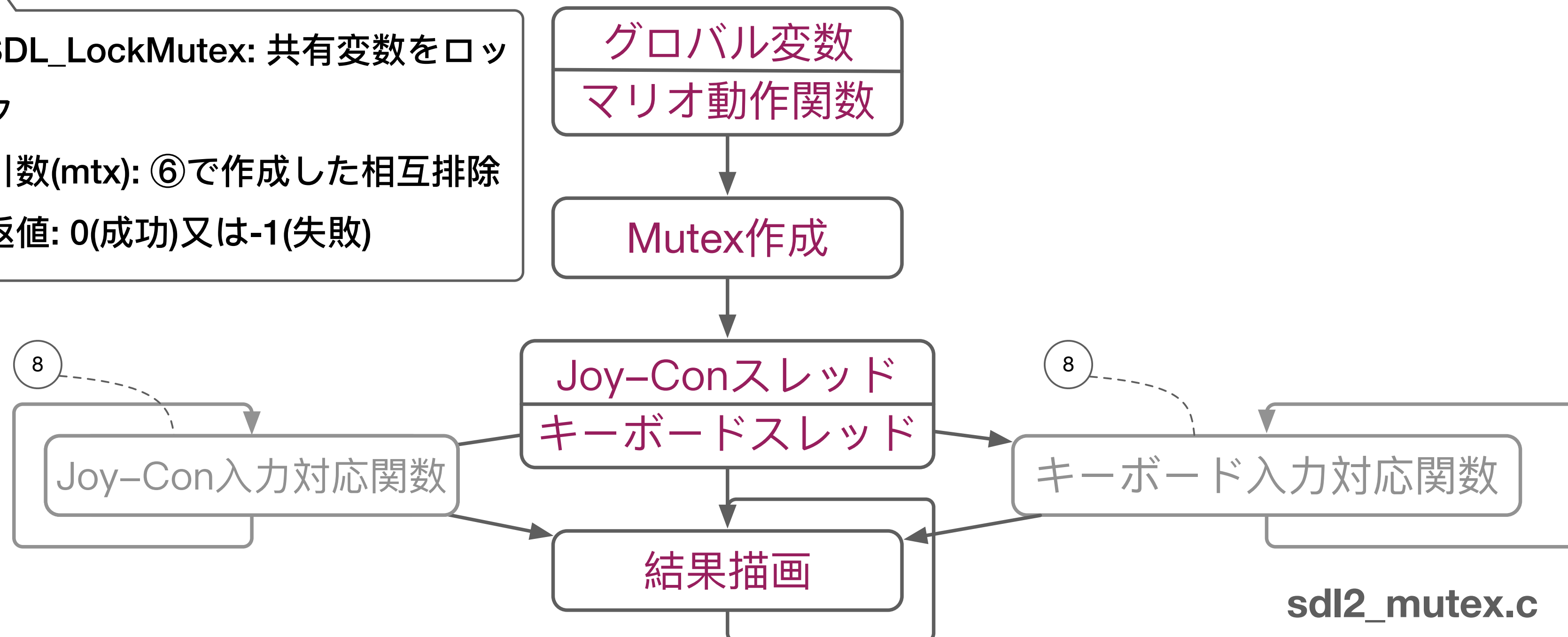


プログラミングの手順

```
////////////////////////////////////  
// STEP. 8                               //  
// STEP_8_BEGINからSTEP_8_ENDまでの//  
// コードのコメントを外す           //  
////////////////////////////////////  
  
// ***** STEP_8_BEGIN *****  
// SDL_LockMutex(mtx); // ミューテックスをロック  
//***** STEP_8_END *****
```

- ⑥ 「Mutex」 作成
- ⑦ 「スレッド関数の引数」 作成
- ⑧ 「mario_loc」 ロック
- ⑨ 「mario_loc」 アンロック
- ⑩ 「Mutex」 破壊

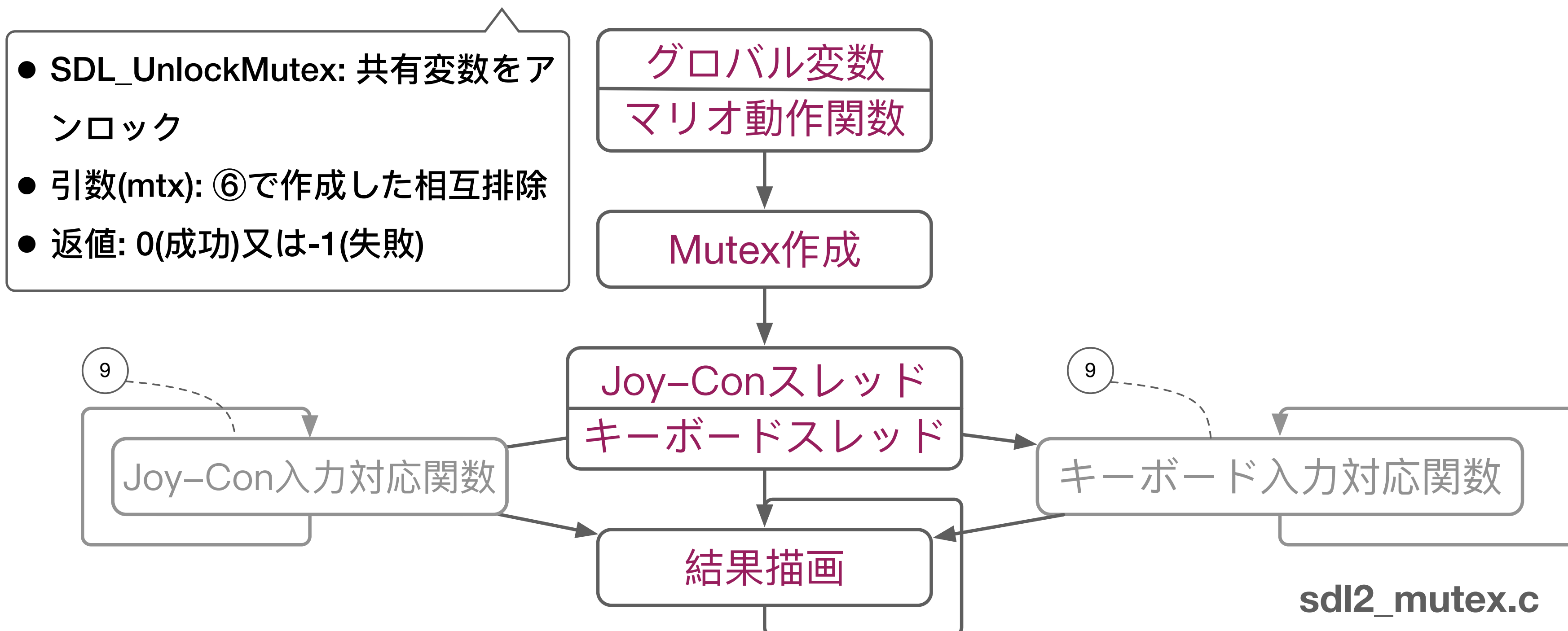
- SDL_LockMutex: 共有変数をロック
- 引数(mtx): ⑥で作成した相互排除
- 返値: 0(成功)又は-1(失敗)



プログラミングの手順

```
////////////////////////////////////  
// STEP. 9                               //  
// STEP_9_BEGINからSTEP_9_ENDまでの//  
// コードのコメントを外す           //  
////////////////////////////////////  
  
// ***** STEP_9_BEGIN *****  
// SDL_UnlockMutex(mtx); // Mutexをアンロックし、他のスレッドが共有変数にアクセスできるようにする  
// ***** STEP_9_END *****
```

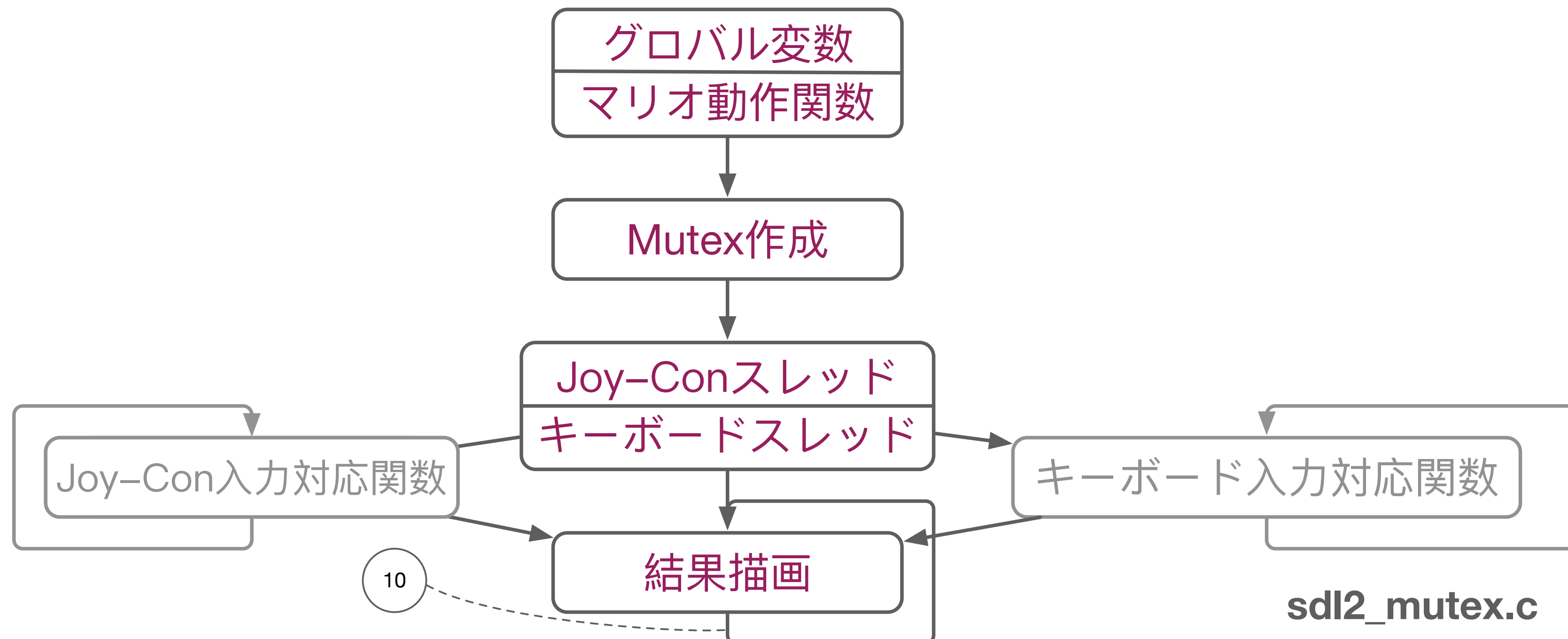
- ⑥ 「Mutex」 作成
- ⑦ 「スレッド関数の引数」 作成
- ⑧ 「mario_loc」 ロック
- ⑨ 「mario_loc」 アンロック
- ⑩ 「Mutex」 破壊



プログラミングの手順

```
////////////////////////////////////  
// STEP. 10                                //  
// STEP_10_BEGINからSTEP_10_ENDまでの//  
// コードのコメントを外す                //  
////////////////////////////////////  
  
// ***** STEP_10_BEGIN *****  
// SDL_DestroyMutex(mtx); // ミューテックスを破棄  
// ***** STEP_10_END *****
```

- ⑥ 「Mutex」 作成
- ⑦ 「スレッド関数の引数」 作成
- ⑧ 「mario_loc」 ロック
- ⑨ 「mario_loc」 アンロック
- ⑩ 「Mutex」 破壊



相互排除のまとめ

- 目的: スレッドで共有変数を独占する為に
- 方法
 - 相互排除変数Mutex作成
 - スレッド関数にMutexを渡す
 - 共有変数をロック
 - **共有変数の処理**
 - 共有変数をアンロック
 - Mutexを破壊

課題

マリオゲームのマルチスレッド化:

1. ソースコード「sdl2_mutex.c」とmakefileを編集
2. makefileの修正「sdl2_thread」→「sdl2_mutex」
3. Step. 1~5に従ってsdl2_mutex.cに「Joy-Con入力対応」スレッドを作成
4. Step. 1~5を参照してsdl2_mutex.cにキーボードを含む「イベント処理」スレッドを作成
5. Step. 6~10に従ってsdl2_mutex.cに「mario_loc処理」相互排除を作成

提出物（学籍番号.zip）：

1. makefile
2. sdl2_mutex.c